Quantum-resistant SPDM device on PSoC Control C3M6

From ECC to CNSA 2.0: ML-DSA-87 verified boot + SPDM 1.4 with ML-DSA-87 / ML-KEM-1024 - on real hardware, measured

SEMICON 2026 PQC Hackathon | KIT_PSC3M6_EVAL | ModusToolbox 3.9, Edge Protect Bootloader, libspdm 4.0

The target: CNSA 2.0, not 'some PQC'

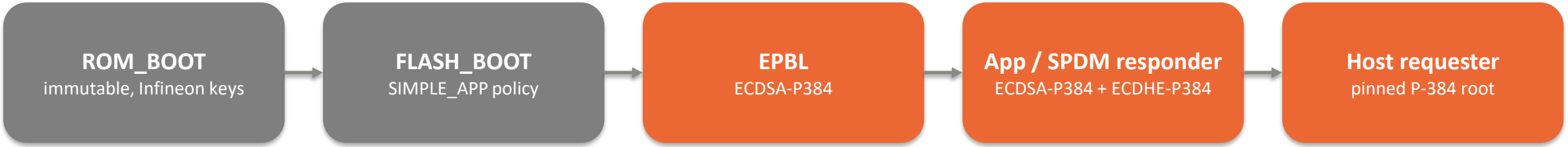
ECDSA and ECDH are quantum-vulnerable; datacenter attestation identities live for 10+ years (harvest-now, forge-later)

Function	Original (ECC baseline)	Migrated	CNSA 2.0 requirement	Enforced / tested
Firmware signature (EPBL -> app)	ECDSA-P384, SHA-384	ML-DSA-87, SHA-512 pre-hash	ML-DSA-87 (or LMS/XMSS)	ECDSA, ML-DSA-65, SHA-256 images rejected
SPDM device authentication	ECDSA-P384	ML-DSA-87 (SPDM 1.4 PqcAsymAlgo)	ML-DSA-87	ML-DSA-65 / P-384 roots rejected
SPDM key establishment	ECDHE-P384	ML-KEM-1024 (SPDM 1.4 KEMAlg)	ML-KEM-1024	no classical algorithm offered
Certificates / trust anchor	P-384 root -> leaf (1.2 KB)	ML-DSA-87 root -> leaf (15.2 KB)	ML-DSA-87 CA	pinned PQC root
Hash / KDF / AEAD	SHA-384, AES-256-GCM	SHA-384/512, AES-256-GCM	SHA-384/512, AES-256	-
BootROM -> first stage	ECDSA (immutable ROM)	unchanged (classical)	-	documented dependency

ML-DSA-65 / ML-KEM-768 are the common commercial default - but category 3, so not CNSA 2.0. CNSA 2.0 is a design constraint here, not a certification claim.

Starting point: the ECC baseline (measured)

Infineon SPDM code example + Edge Protect Bootloader, DEVELOPMENT lifecycle



Baseline measurement	Value
ECDSA-P384 signature (device)	84.4 ms
ECDHE-P384 keygen + shared secret	162.9 ms
Session setup (device time)	~257 ms
EPBL image verification	~170 ms
Cert chain / largest message	1,218 B / 1,226 B
Session: transfers / bytes	24 / 2,569 B

Baseline tests	Result
Authorized image boots	PASS
Corrupt / wrong key / header / TLV / signature	REJECTED
Rollback (v0.9 update)	REJECTED
SPDM suite (9 ops)	PASS
ECC tamper builds (cert, sig, transcript, MITM)	6/6 REJECTED
Wrong trust anchor	REJECTED

Migrated architecture: one PQC chain, reset to session

The SPDM responder is the image EPBL verified - and its signed measurements prove it



512 KB flash map



EPBL 108 KB | PAS 24 KB: ML-DSA-87 key (on-chip keygen) + 15.2 KB cert chain - outside both slots, survives every update

- Boot plane: EPBL verifies ML-DSA-87 (HashML-DSA, SHA-512) - 80 ms for the 165 KB responder
- Runtime plane: SPDM 1.4 - ML-DSA-87 identity, ML-KEM-1024 session keys, AES-256-GCM; no classical algorithm offered
- Link: signed measurements = EPBL digest + verified-image digest + version, recomputed by the host

Boot-time protection: EPBL with ML-DSA-87

BootROM is NOT assumed to verify ML-DSA - EPBL is the PQC enforcement point

Test (EPBL, ML-DSA-87)	Demo app (T2)	SPDM responder (T4)
Authorized image boots	PASS	PASS
Valid signed update installs	PASS	PASS (v1.1, v2.0)
Corrupted payload	REJECTED	REJECTED
Modified header / protected TLV	REJECTED	REJECTED (header)
Rogue ML-DSA-87 key	REJECTED	REJECTED
Rollback to older signed version	REJECTED	REJECTED
Signature byte / malformed TLV	REJECTED	-
Classical ECDSA image (downgrade)	REJECTED	-
ML-DSA-65 / SHA-256 pre-hash (non-CNSA)	REJECTED	-
Tampered installed image	no boot	no boot

61 ms

ML-DSA-87 image verification (ECDSA-P384: 170 ms)

47 KB

EPBL flash (+11 KB vs ECDSA), peak boot RAM 25 KB

4.95 s

install a 180 KB responder update (2 verifications)

Real-time protection: SPDM 1.4 with ML-DSA-87 and ML-KEM-1024

Same responder code, profile P2 -> P4; the requester validates a 15 KB ML-DSA-87 chain against a pinned root

Negative test (12/12 rejected)	Rejected at
Classical P-384 / rogue / ML-DSA-65 root	get_certificate
Malformed / truncated root	get_certificate
Leaf certificate modified on device	get_certificate
ML-DSA signature byte flipped	challenge
Valid signature, modified transcript	start_session
ML-KEM ciphertext modified	start_session (key confirm.)
MITM: AEAD tag of FINISH_RSP	start_session
MITM: AEAD tag of app data	send_receive

- **Session secret**
  - Host: ephemeral ML-KEM-1024 key (1,568 B) in KEY_EXCHANGE
  - Device SPE: Encaps -> ciphertext + 32 B secret (11 ms)
  - HKDF-SHA-384 key schedule bound to the transcript; FINISH confirms
  - Fresh keys per session, no long-term KEM key on the device
- **Positive**
  - connect, certificate, challenge, measurements, session, encrypted exchanges: PASS
  - 1 one-off transport abort (failed closed, did not recur in any later handshake)
- **Memory model**
  - all buffers static & size-checked: 16 KB chain copy, 35.8 KB scratch, 7 KB max message, transcript as running hash

Lifecycle: update, rollback, credentials - one chain of trust

Task 4: the real responder updated through EPBL, reconnecting over SPDM after every step (18/18 as expected)

v1.0 boot
SPDM PASS

update v1.1
installed, SPDM PASS

rollback v1.0
REJECTED

corrupt / header / rogue key
REJECTED

tamper primary
NO BOOT

update v2.0
recovered, SPDM PASS

measurements after every boot: EPBL digest, image digest, version = values recomputed from the release files (4/4 MATCH)

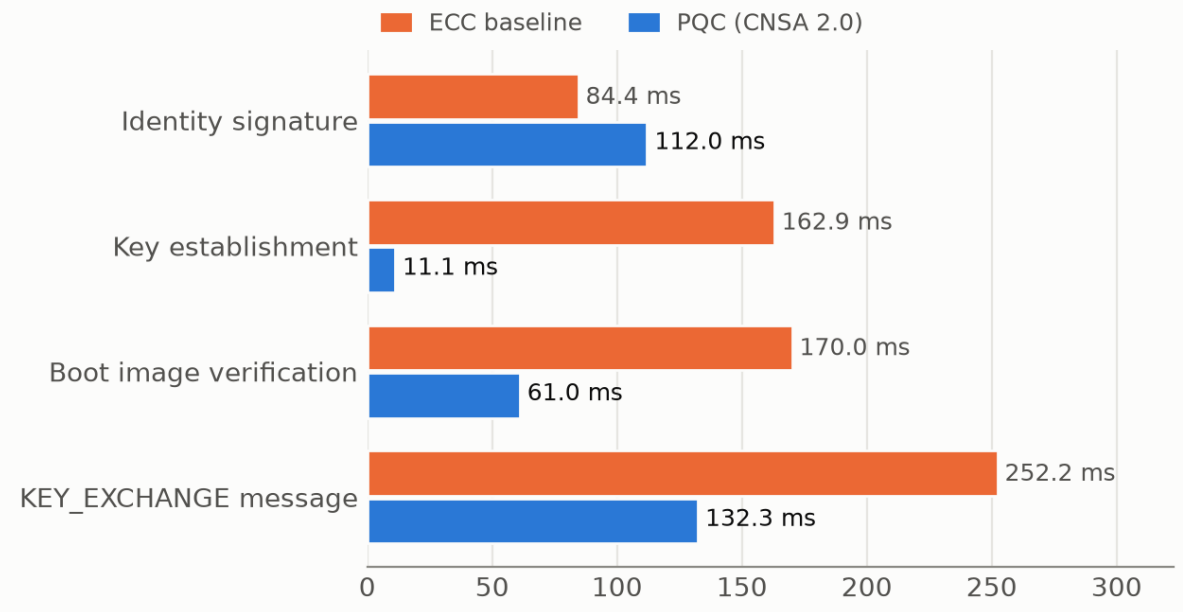
Credential path (PAS)	Result
Rotation: new on-chip ML-DSA-87 key + cert, reconnect	PASS
ML-DSA-65 / ECDSA keygen	REJECTED
Chain larger than the 15 KB slot	REJECTED
Certificate for a foreign key	proof-of-possession FAILS
SHA-256 RootHash vs negotiated SHA-384	REJECTED by host
Key overwrite before freeze (V5b)	accepted - by design, needs freeze

Identity question	Our answer
Key anchored in	separately provisioned PAS credential
Firmware state anchored in	verified boot -> signed measurements
Update changes identity?	no - reconnect without re-provisioning
Unsigned image uses the key?	cannot run (EPBL)
Remote rollback detection	version in measurement block 3
Next step	DICE-style key derived from image

What it costs: measured ECC baseline vs PQC

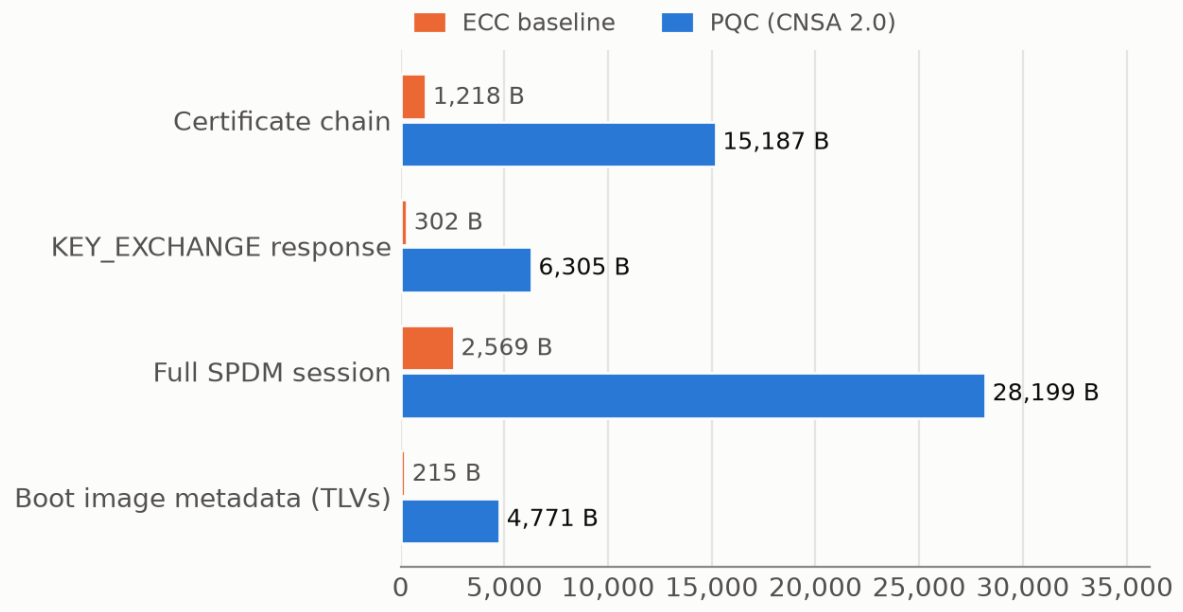
Same board, same code base, Release builds; device-side timings from on-chip instrumentation

Device time per operation (ms) - lower is better



ECDSA-P384 / ECDHE-P384 (SW mbedTLS) vs ML-DSA-87 / ML-KEM-1024 (wolfCrypt). ML-DSA signing is rejection-sampled: 66-346 ms, median shown.

Bytes on the wire / in flash per object

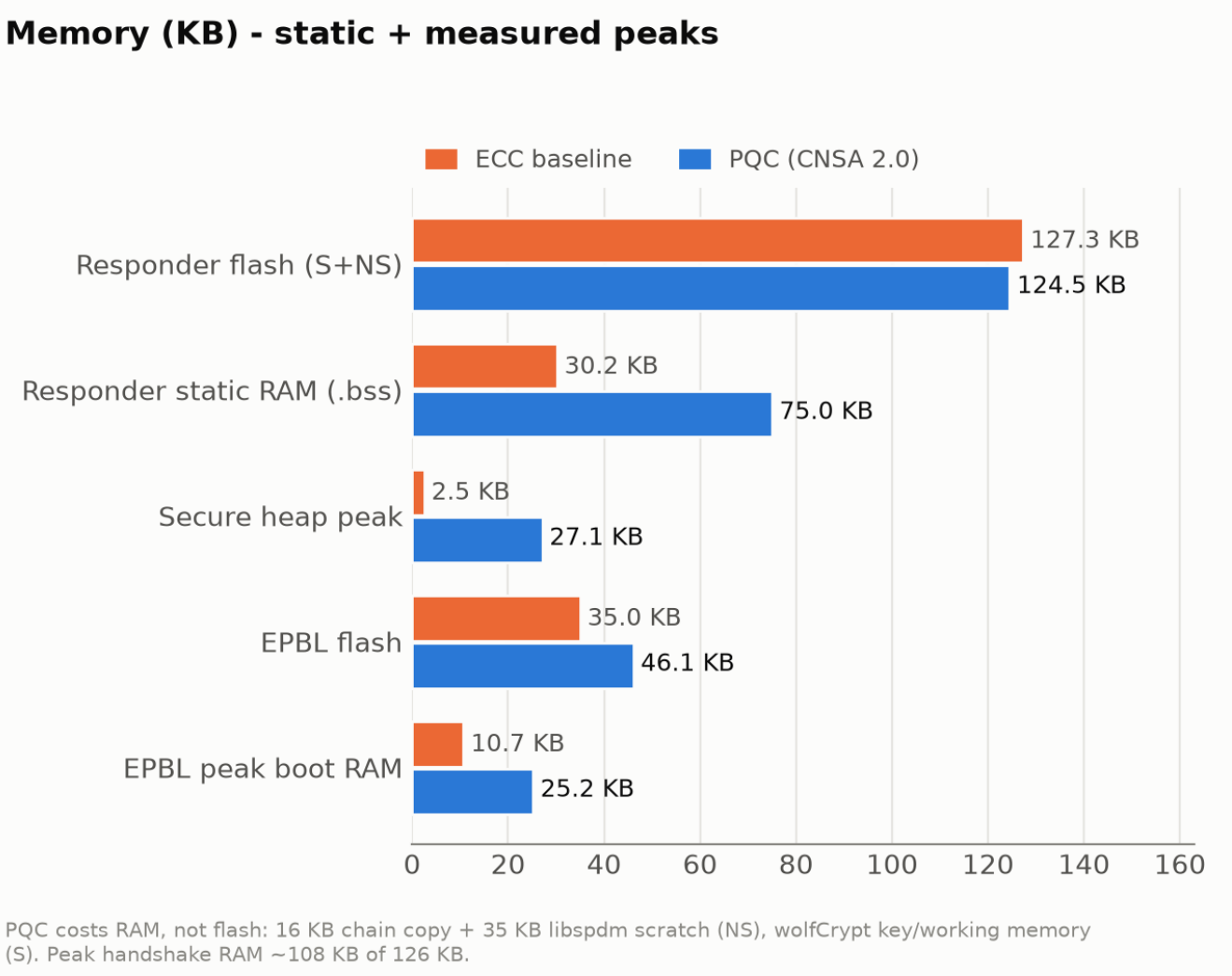


Session = VERSION ... cert fetch ... KEY_EXCHANGE / FINISH / END_SESSION over 127 B DFU transfers (24 -> 214 transfers).

- Faster where it matters: ML-KEM 15x faster than SW ECDH, ML-DSA verify 3x faster than ECDSA
- Bigger everywhere: chain x12.5, session bytes x11, 24 -> 214 transfers over 127 B DFU - host time 38 s -> ~6 min per session

What it costs: memory

PQC moves the cost from flash to RAM



PQC costs RAM, not flash: 16 KB chain copy + 35 KB libspdm scratch (NS), wolfCrypt key/working memory (S). Peak handshake RAM ~108 KB of 126 KB.

~108 KB

peak handshake RAM of 126 KB (ECC: ~40 KB)

15,187 B

ML-DSA-87 chain in a 15,360 B PAS slot (99 % full)

4,771 B

boot metadata per image (ECDSA: 215 B)

What is still classical - and what must change

We do not claim full quantum resistance yet; here is exactly what is missing

Item	Status today	Required before claiming full quantum resistance
ROM_BOOT / FLASH_BOOT -> EPBL	classical ECDSA; not enabled on the shared board	OEM key + SECURE_APP policy now; PQC-capable ROM long term
EPBL key rotation	EPBL not field-updatable in this map	BootROM-authenticated EPBL update path
Rollback	software downgrade prevention + version in SPDM measurements	eFuse anti-rollback counter + closed DAP
Credential store	not frozen; provisioning channel unauthenticated	freeze after provisioning; authenticated re-provisioning
Debug / lifecycle	DEVELOPMENT, debug port open	SECURE lifecycle, DAP closed
Update delivery	debugger writes the upgrade slot	in-band update over the SPDM transport
Transport	127 B DFU transactions over I2C	MCTP over I3C / PCIe VDM, chunk retry
SPDM algorithms	PQC only (ML-DSA-87, ML-KEM-1024)	done

Migration path for devices already in the field with ECC

No hybrid (per brief): each plane is cut over per device; the fleet accepts both roots only during the transition

1	**Firmware update to a PQC-capable image** through the existing ECDSA-verified EPBL update path	**shown (Task 1/2)**
2	**EPBL validation key ECDSA -> ML-DSA-87** production: BootROM-authenticated EPBL update	**shown by reflash; BootROM path documented**
3	**On-chip ML-DSA-87 key + CSR, signed by a new ML-DSA-87 OEM CA** production: CSR delivered over a session authenticated by the old ECC identity	**keygen + CA shown; ECC bridge documented**
4	**Responder switches to the PQC-only SPDM profile** no classical algorithm offered -> no downgrade	**shown (Task 3/4)**
5	**Requesters drop and revoke the ECC root** fleet policy	**classical root rejected - shown**
6	**Freeze credentials, close debug, SECURE lifecycle, eFuse counter** irreversible	**documented (shared board)**

Lessons learned and why it matters beyond the hackathon

Lessons

- Check the parameter set: CNSA 2.0 = category 5 (ML-DSA-87, ML-KEM-1024), not the commercial default
- PQC compute is not the bottleneck on Cortex-M33 - bytes, RAM and the transport are
- Defaults bite: EPBL ML-DSA expects SHA-512 pre-hash, the signer defaulted to SHA-256
- Attestation must be bound to verified boot - we replaced placeholder measurements with EPBL-verified values
- Test the negatives on the baseline too - fair comparison, no false confidence

Market and route to product

- Every SPDM responder (GPU, SmartNIC/DPU, SSD controller, CXL) needs this migration; long service life
- CNSA 2.0 timelines drive U.S. national-security suppliers first; CRA/NIS2 push lifecycle readiness in the EU
- Deliverable: migration kit on PSoC Control C3 - EPBL + SPDM integration, provisioning, test harness, measured budget
- Adoption cost is low: same responder code, build-time profile, no new silicon for the PQC plane
- Next: DICE-bound keys, in-band updates, faster transport, production lock-down

Live demo (5 min)

Board: EPBL (ML-DSA-87) + SPDM responder; host: requester over I2C

- 1 Verified boot: reset -> EPBL verifies the responder with ML-DSA-87 -> SPDM responder starts
- 2 Tampering: update signed with a rogue ML-DSA-87 key -> rejected, old version keeps running
- 3 Rollback: older genuine image as update -> rejected (insufficient version)
- 4 Valid signed update v1.1 -> installed in ~5 s -> responder restarts with the same identity
- 5 SPDM after update: ML-DSA-87 authentication, ML-KEM-1024 session, encrypted message; signed measurements = release digest

- SPDM over this 127 B I2C tunnel takes ~6 min per session -> started ahead of time; recorded run as backup